

Introduction

In this assignment, we identified several code smells in our project and refactored them to make the code cleaner, easier to read, and easier to maintain. Below each smell found, how we fixed it, and why the refactoring improved the program has been described. All refactored code was tested, committed, and pushed after each change.

1. In the initial code, player movement direction was represented as a string with a special "magic string" and it created a "primitive obsession" code smell. This technique was not type-safe, it created an opportunity for invalid values, and it reduced maintainable code. In the revised design, there is a Direction enum created with all string equals replaced by enum constants. This prevents bugs based on typos and consolidates the definitions of direction, making for cleaner code while facilitating easier extension down the line. Furthermore, by moving direction-based behavior into the enum, we reduced conditional behavior while increasing cohesion.
2. We previously used raw strings like "up", "down", "left", and "right" for movement in both Player and Zombie, which created a primitive obsession and magic-string smell. This caused issues with type safety, inconsistent values, and duplication across classes. We refactored both entities to use a shared Direction enum, eliminating typos and centralizing the definition of valid directions. Movement and rendering now rely on a single enum-based switch, making behavior more consistent between Player and Zombie. Overall, this refactoring improves readability, prevents invalid states, reduces duplication, and makes the codebase more maintainable.
3. A number of Zombie fields (xPos, yPos, speed, solidArea, direction) were originally public or had package visibility, meaning that external manipulation was easily accessible and encapsulation was broken. This was an example of a typical encapsulation smell. We fixed this by making these fields private (and final where applicable) while giving access to only the number of public getters and the existing method setSpeed. Other classes now interact with Zombie through its public API and no longer can change its fields through direct access, promoting encapsulation and rendering the class easier to reason about for maintenance.
4. Many fields in the original Player class were public/package visible (direction, spriteNum, speed, solidArea, collisionOn, and sprite images). This means that the Player's internal state could be mutated freely by other classes, most notably, GamePanel. This is how encapsulation was broken. We repaired this by making these fields private (and final where applicable) and rendering a number of accessors available to interact with these fields (getSolidArea(), getSpeed(), setDirection(...)). Now GamePanel can interact with Player through its public API without directly manipulating the values it has previously had access to, improving encapsulation and enabling a clearer perspective on how Player works.

5. In the Coin class, the loadCoinImage() method originally had duplicated code because every frame of the coin animation was being loaded one by one manually. This is a duplicated code smell. To fix this, I replaced all the repeated lines with a loop that loads each sprite using an index. This removes all the duplication and makes the method easier to update in the future if the number of images changes.
6. The Player class had a long method inside checkCollision(), where all four directions were handled in one large chunk of code. This is a long method smell because it becomes hard to understand and hard to modify. I refactored it by splitting the logic into separate helper methods: checkRightCollision(), checkLeftCollision(), checkUpCollision(), and checkDownCollision(). This makes the code easier to read, easier to debug, and follows the single-responsibility principle.
7. In the Player Draw Method we replaced Switch Statement Replaced with Polymorphism In the draw() method of the Player class, there was a large switch statement that selected the sprite depending on the direction and sprite number. This is a switch statement smell and also caused duplicated code. I replaced the switch with polymorphism by creating a Direction enum where each direction (UP, DOWN, LEFT, RIGHT) knows how to return its own sprite. Now the draw method only calls direction.getSprite(this). This removes duplication, makes the method shorter, and makes it easier to add new directions or animations later.
8. In the KeyHandler class, both keyPressed() and keyReleased() had repeated if-statements for every key. This is another duplicated code smell. I fixed this by adding a new method called updateKeyState(KeyEvent e, boolean pressed). Both keyPressed() and keyReleased() now call this method. Inside updateKeyState, a single switch statement updates the correct boolean value. This makes the class more compact, easier to update, and avoids repeating the same logic twice

Conclusion

In summary, the refactoring gave the code base a clean appearance, increased safety and maintainability. We removed magic strings by means of a shared Direction enum; increased encapsulation in Player and Zombie; decreased duplicated logic in Coin and KeyHandler; split long methods into smaller ones; and changed switch statements into polymorphism. All of these implementations helped us eliminate some major code smells and solidified the project in its design. It read easier, had less potential bugs, and making changes down the road for expansion (new features and/or new animations) will be far easier. We also assessed that nothing changed behaviourally during the process as we ran our tests multiple times through the analysis and made a separate commit for each change and we ended up with a more compartmentalized, extensible and reliable code base going forward.